

INTERNSHIP SELECTION ASSIGNMENT

AI & Data Science Track

Agentic RAG over Mixed Data Sources

Candidate Copy - Do Not Distribute

Time to Complete	10 Calendar Days from date of receipt
Submission Format	GitHub repo + CLI/notebook demo + design document + evaluation set
Core Evaluation Areas	Tool Design · Agent Reasoning · Answer Quality · Safety & Termination · Code Quality
Post-Submission	Top 5 candidates run their agent live during a 45-minute interview and walk through one failure case

1. Welcome & Purpose

Congratulations on reaching the assignment stage of our internship selection. This problem moves beyond single-dataset analysis into something closer to real production AI work: building a small reasoning agent that decides which tool to call, in what order, and then composes a coherent answer from multiple sources.

You are not being asked to wrap a library and call it a day. We want to see you design and build the core agent loop yourself - so that you can read it top to bottom and explain every decision it makes.

What this assignment is really about

- Can you design clean, focused tool contracts that an LLM can reliably call?
- Does your agent pick the right tool for the question - or does it always default to the same one?
- Can it compose an answer that draws on two different sources and cites both?
- Does it know when to stop - and when to admit it cannot answer?
- Can you write an honest evaluation that shows both where it works and where it fails?

2. The Problem

Background

Many real questions cannot be answered from a single source. Consider: "How did Infosys' operating margin compare with TCS' in FY24, and what was the main reason each company gave?" - that requires pulling numbers from a structured financial table and explanations from a management commentary document, then combining both into one answer.

A single RAG pipeline cannot do this well on its own. You need an agent that can look at a question, decide what kind of source will best answer it, retrieve from that source, and then decide whether it needs to look somewhere else before composing the final answer.

Your Assignment

Build a small LLM agent that answers questions over a mixed corpus of documents, structured data, and live web search. The agent must pick the right tool for each question, reason across multiple tools when needed, return a grounded answer with citations, and stop gracefully when it cannot help.

Important framing

We are not evaluating whether your agent achieves state-of-the-art accuracy.

We are evaluating whether you understand what your agent is doing, why it makes the tool choices it does, and where it breaks down.

An agent that you can fully explain and that fails gracefully will score higher than a black-box wrapper that happens to answer most questions.

3. Data Setup

Choose One Corpus

Pick exactly one of the three setups below. All three are equivalent in difficulty. Choose the one you find most interesting - you will be spending 10 days with it.

Option A - Public Company Financials (Recommended for beginners)

- 3 Indian IT companies: Infosys, TCS, and Wipro (or any 3 public companies you prefer).
- Unstructured: Last annual report PDF for each company (free from their investor relations websites).
- Structured: A CSV you build yourself with 4 years of key financials per company - revenue, operating margin, net profit, EPS, headcount. Use published annual reports to fill in the numbers.
- Web: Live web search for recent news, analyst ratings, and current stock prices.

Option B - Sports Season Data

- 2 full seasons of any major league (IPL, Premier League, NBA, etc.).
- Unstructured: Wikipedia match reports or official PDF season reviews.
- Structured: A CSV of per-match stats (goals, scores, player stats) from a public dataset such as Kaggle or football-data.co.uk.
- Web: Live web search for recent transfer news, standings, and injury updates.

Option C - Movies Corpus

- A small corpus of 10–15 long-form film reviews from reputable publications.
- Unstructured: The review texts as PDFs or plain text files.
- Structured: A CSV of box-office data (budget, opening weekend, worldwide gross, Rotten Tomatoes score) from Box Office Mojo or similar.
- Web: Live web search for recent awards news and director updates.

Can I use a different corpus?

Yes, if you can justify that it meets all three requirements: unstructured text (at least 3 documents), a structured table (at least 50 rows and 5 columns), and a web search use case.

State your choice in your README and confirm it satisfies all three before starting.

4. What to Build

The Three Tools

Your agent must expose exactly three tools. Each tool should do one thing well. Well-named, well-described tools are half the battle - the LLM uses your tool descriptions to decide which one to call.

Tool Name	Purpose	Input	Expected Output
<code>search_docs</code>	Semantic search over unstructured documents	Natural language query string	Top-3 relevant text chunks with source filename and page number
<code>query_data</code>	Query the structured financial / stats table	A pandas or SQL query, or a natural-language question about the data	A table or scalar value with column names and row count
<code>web_search</code>	Search the live web for recent information	A short search query string (under 10 words)	Top-3 result snippets with URL and publication date

Each tool must have a clear name, a one-paragraph description written for the LLM (not for a human reader), typed inputs, and a consistent output format. You will be asked to show your tool definitions during the interview.

The Agent Loop

Your agent loop is the core of this assignment. It must be implemented as code you wrote - not a black-box framework call. The loop must:

1. Receive a user question.
2. Decide which tool to call first (or decide no tool is needed).
3. Call the tool, receive the result, and add it to the agent's context.
4. Decide whether the result is sufficient to answer, or whether another tool call is needed.
5. Repeat steps 2–4, up to a maximum of 8 tool calls per question. Enforce this hard cap in code.
6. Compose the final answer, citing exactly which tool calls produced the information used.
7. If the question cannot be answered after 8 steps, return a clear refusal - not a guess.

On the hard cap

The 8-tool-call limit is not a suggestion. Your code must raise an exception or return a structured refusal if this limit is hit. During the interview, we will deliberately ask questions designed to send the agent into a loop. We want to see that the cap fires correctly.

Trace Logging

Every agent run must produce a trace: a structured log of every tool call made, the input sent, the output received, and the final answer. The trace is your primary debugging and evaluation tool.

Minimum trace output format for each question:

```
Question:      [user question]
Step 1:        tool=search_docs  input='operating margin FY24'
               result=[chunk text, source, page]
Step 2:        tool=query_data   input='SELECT op_margin FROM financials WHERE year=2024'
               result=26.3% (Infosys FY24)
Final Answer:  [composed answer]
Citations:     search_docs (Infosys AR FY24, p.47), query_data (financials.csv row 12)
Steps used:    2 / 8 max
```

5. Detailed Task Breakdown

Task A - Setup & Tool Scaffolding (Days 1–2)

Before writing any agent logic, get all three tools working independently and verifiably. This phase tests your ability to scaffold a system cleanly before connecting the parts.

Deliverable A: A GitHub repository with:

8. All 3 tools individually callable from the command line or a notebook. Each tool should work and return results before the agent loop exists.
9. Your vector store indexed with the documents from your chosen corpus. Verify retrieval quality manually: run 5 queries and check that the top chunk returned is actually relevant.
10. Your SQLite database or CSV loaded and queryable. Confirm that you can answer at least 3 structured questions correctly by hand before handing it to the agent.
11. A .env.example file showing every secret your project needs (API keys, etc.) with placeholder values. No real keys in any commit, ever.
12. A requirements.txt or environment.yml that allows someone else to reproduce your environment in one command.

Why start with tools, not the agent?

Agents built on top of broken tools are very hard to debug. A tool that silently returns wrong results will mislead your agent in subtle ways.

Validate each tool independently first. Only connect them to the agent once you trust each one.

Task B - Agent Loop & Single-Tool Questions (Days 3–4)

Implement the core agent loop and verify it on questions that require only one tool. This phase tests your understanding of how LLM tool-calling works under the hood.

Deliverable B: An agent that:

13. Takes a question as input and returns a final answer with trace.
14. Correctly routes to search_docs for questions about document content.
15. Correctly routes to query_data for questions about structured numbers.
16. Correctly routes to web_search for questions about recent or live information.
17. Correctly returns a direct answer (no tool call) for trivial questions like “what is 2+2”.
18. Hard cap of 8 tool calls is implemented and tested.

Run your agent against at least 10 questions from your evaluation set and record which ones it gets right, which ones it calls the wrong tool for, and which ones it fails to answer. Be honest in your notes.

Task C - Multi-Tool Reasoning & Citation (Days 5–7)

Extend the agent to handle questions that require combining information from two or more tools. This is the hardest phase and the most differentiating one.

Deliverable C: An agent that can:

19. Identify when a question requires more than one tool (e.g., a number from the CSV and an explanation from the document).
20. Make the first tool call, assess whether the result is sufficient, and then decide whether to call a second tool.
21. Compose a final answer that draws on both sources and clearly attributes which part came from which tool.
22. Produce a citation list that names the exact source used for each claim in the answer (not just “search_docs was called” - the actual document name, page, or row).

Test on at least 5 multi-tool questions from your evaluation set. Record the traces. Do not cherry-pick - include the failures.

Task D - Safety, Refusals & Evaluation Set (Days 8–9)

Add guardrails and build your evaluation set. This phase tests your understanding of where your agent breaks down and your honesty in reporting it.

Safety & Termination:

23. Enforce the 8-step hard cap with a clear error message or structured refusal response.
24. Add at least one question to your eval set that is deliberately designed to trigger a loop (a question with no good answer in any of your sources). Verify the cap fires.
25. Add at least one question that should be refused entirely (e.g., “which company should I invest in?”). Verify your agent refuses it without calling any tool.
26. Add at least one question that is unanswerable from your data (e.g., asks about a period not covered by your corpus). Verify the agent says so honestly rather than hallucinating.

Evaluation Set:

Build a set of at least 20 questions covering the four categories below. Your evaluation document should include the question, the expected answer or expected behaviour, the tool(s) expected to be called, and the actual output from your agent.

- Single-tool questions (at least 6): answerable from exactly one tool.
- Multi-tool questions (at least 6): require combining two or more tools.
- Refusal questions (at least 4): questions the agent should decline to answer.
- Edge cases (at least 4): ambiguous questions, questions at the boundary of your data, or questions designed to expose weaknesses.

Task E - Final Packaging & Submission (Day 10)

Deliverable E: Your complete submission package:

27. GitHub Repository - public, with a clean commit history showing daily progression. All code must run top-to-bottom without errors.
28. README - written for an engineer, not a marketer. Include: what the project does, how to set it up, how to run the agent, how to run the evaluation set, and what the known failure modes are. Honest READMEs score higher than promotional ones.
29. Design Document (1–2 pages) - describe your agent loop step by step, your tool schemas (name, description, input, output), and exactly how you prevent infinite loops. This should be readable by someone who has not seen your code.
30. Evaluation Report - your 20-question eval set with actual agent outputs, accuracy by category, and a written reflection on at least 2 failure modes you observed.
31. Demo - a short screen recording (≤5 minutes) or a GIF walking through 2–3 representative agent traces: one success, one multi-tool trace, and one graceful failure or refusal.

6. Example Evaluation Questions

The table below shows 10 example questions for the public company corpus (Option A). If you chose Option B or C, adapt the spirit of these questions to your corpus - ensure you cover all four categories.

#	Example Question	Tools Required	Expected Outcome
1	What was Infosys' operating margin in FY24?	SQL / CSV tool	Single number with source
2	What reason did TCS give for its margin improvement in FY24?	RAG / doc search	Quoted explanation with citation
3	How did Infosys' and TCS' operating margins compare in FY24, and what drove each?	SQL + RAG (multi-tool)	Composed answer from both tools
4	What is the current stock price of Infosys?	Web search	Live price with source URL
5	Who is the current CFO of TCS?	Web search or RAG	Name with source
6	What was Wipro's revenue growth over the last 4 years?	SQL / CSV tool	Year-by-year table
7	What strategic priorities did Infosys highlight in their FY24 MD&A?	RAG / doc search	Bullet points with citation
8	Compare headcount growth at all 3 companies over 4 years.	SQL + RAG (multi-tool)	Comparison with sources
9	What happened to IT sector stocks last week?	Web search	Recent news summary
10	What is the airspeed velocity of an unladen swallow?	None - refuse	Polite refusal, no tool called

On the evaluation set

Do not fabricate expected answers. Run each question against your agent, record the actual output, and then judge it honestly.

A submitted evaluation that shows failures and explains why they happen is worth far more than one that only shows successes.

7. Evaluation & Scoring

Each submission is reviewed by two evaluators. The top 5 scorers are invited for a 45-minute live session where you run the agent in real time, explain your design choices, and walk through one tool-choice mistake your agent made.

Criterion	Weight	What We Look For
Tool design	20%	Clear tool contracts: name, description written for the LLM, typed inputs, consistent output schema. Each tool does one thing well and nothing else.
Agent reasoning	25%	Agent picks the right tool for the question, calls tools in a sensible order, and composes information across tools rather than just concatenating outputs.
Answer quality	20%	Answers are correct, grounded in the sources actually retrieved, and cite the specific document/row/URL used. No hallucinated citations.
Safety & termination	15%	Hard cap enforced in code, refusal questions handled gracefully, agent does not loop or guess when it should stop.
Evaluation & reflection	10%	Evaluation set covers all four categories, actual outputs are recorded honestly, and at least two failure modes are discussed with insight.
Code quality	10%	Code is readable without comments to explain it, secrets are in .env and never committed, project reproduces in one setup command.

Scoring bands:

- 85–100: Strong hire - invited to final round
- 70–84: Potential hire - invited with specific follow-up questions
- 55–69: Hold - considered if top candidates decline
- Below 55: Not proceeding

Automatic disqualification triggers

- Agent loop implemented entirely by calling `initialize_agent()` or equivalent without being able to explain its internals
- Any secret (API key, password) committed to the repository at any point in the commit history
- Hard cap not enforced in code - agent can loop more than 8 times
- Code does not run end-to-end without modification on a clean install

8. Suggested 10-Day Timeline

The Day 10 submission deadline is firm. The timeline below is a recommendation - adapt it to how you work. What we care about is that each phase is genuinely complete before you move to the next.

Phase	Days	Focus	Deliverable
Phase 1	1–2	Environment setup, data ingestion, and tool scaffolding	Working repo with all 3 tools callable individually
Phase 2	3–4	Agent loop, tool selection logic, and trace logging	Agent answers simple single-tool questions with trace
Phase 3	5–7	Multi-tool reasoning, answer composition, and citation	Agent handles multi-source questions end-to-end
Phase 4	8–9	Safety guardrails, evaluation set, and honest failure analysis	20-question eval set + termination enforcement
Phase 5	10	README, design doc, demo, and submission packaging	Full GitHub repo + design doc + video/GIF

Common mistake: skipping to the agent loop too early

Candidates who start building the agent loop on Day 1 consistently spend Days 3–5 debugging tool failures that should have been caught on Day 1.
Trust the timeline. Get all three tools working and manually verified before writing a single line of agent logic.

9. Recommended Tech Stack

You are free to use any stack that meets the requirements. The guidance below is what we have seen work well for this problem at intern level.

LLM with tool / function calling

- OpenAI GPT-4o or GPT-4o-mini (function calling, reliable tool routing, good JSON output).
- Anthropic Claude via the API (tool use API, strong reasoning, good at refusing).
- Either works well. Pick the one whose API you are more comfortable with.

Vector store for document search

- FAISS - lightweight, no server needed, good for a corpus under 1,000 chunks.
- Chroma - slightly easier API, good persistence, works locally.
- Either is fine. Do not use a hosted cloud vector DB for this assignment - keep it local and reproducible.

Structured data

- SQLite + the sqlite3 Python library - simple, file-based, no setup needed.
- pandas with a CSV - even simpler. Let the agent generate pandas queries or translate to plain lookups.
- Either is acceptable. SQLite is slightly preferred because the tool contract is cleaner (SQL in, result out).

Web search

- Tavily API - designed for LLM agents, returns clean snippets with URLs. Free tier is sufficient.
- SerpAPI or DuckDuckGo Search - both work. SerpAPI has a free tier.
- Do not use a browser automation tool (Selenium, Playwright) for web search - it is overkill for this task and expensive in time.

Agent loop

- Implement it yourself as a Python function or class. A plain while loop with a step counter is completely fine.
- You may reference LangGraph or ReAct paper for ideas on loop design, but do not use high-level agent wrappers ("initialize_agent") as a black box.
- Keep your loop under 100 lines of Python. If it is longer, you have over-engineered it.

On API cost

Keep your total API spend under ₹1,500 (approximately \$20 USD) for the whole assignment.

Cache LLM responses during development using a simple dict or a file-based cache. Do not call the LLM repeatedly for the same test question.

Use a smaller model (GPT-4o-mini, Claude Haiku) for development and only switch to a larger model to verify final results.

10. How to Submit

32. GitHub Repository - public repo, clean commit history, working from a fresh clone.
33. README - setup instructions, run command, known failures. Treat the reader as an engineer.
34. Design Document - included in the repo as DESIGN.md or as a PDF.
35. Evaluation Report - included in the repo as EVALUATION.md or as a PDF.
36. Demo Recording - upload to Loom / YouTube (unlisted) / Google Drive. Set to “Anyone with the link.”
37. Submission Form - fill out the form in your assignment email with all links. This is your official submission trigger.

Before submitting, run through this checklist:

- Fresh clone of the repo + one setup command → agent runs without errors
- No API keys or secrets anywhere in the git history
- Hard cap is enforced and tested with at least one loop-inducing question
- Evaluation set has at least 20 questions across all four categories
- Design document explains the agent loop and tool schemas clearly
- Demo recording shows at least one multi-tool trace and one graceful failure

11. Rules & Academic Integrity

- This is an individual assignment. Do not collaborate with other candidates.
- You must implement your own agent loop. You may reference open-source examples for learning, but the code you submit must be yours and you must be able to explain every line live.
- You may use AI coding assistants (GitHub Copilot, ChatGPT, Claude). Disclose this in your README and be prepared to explain all design decisions in the live review.
- Do not commit any API key, password, or secret to your repository at any point - including early commits. Check your git history before submitting.
- You may send one clarification email to the hiring contact. We will answer factual questions about the requirements but will not provide implementation hints.
- Do not share this assignment brief or your work publicly until after the selection process closes.

12. Bonus Challenges (Optional)

These are optional. Completing any of them will improve your score but none are required. Do not attempt bonus challenges at the expense of completing the core tasks.

- **Bonus A:** Planning step - Before calling any tool, have the agent write a short plan (1–3 sentences) describing which tools it intends to use and why. Measure whether this improves accuracy on your evaluation set and report the result.
- **Bonus B:** Per-tool telemetry - Log latency, estimated token cost, and call count for each tool per question. Produce a small summary table in your evaluation report showing which tool is most expensive and which is called most often.
- **Bonus C:** Reflection step - After producing the final answer, add a step where the agent critiques its own answer: does it actually answer the question asked? Is every claim cited? If not, trigger one more retrieval round. Show before/after accuracy.
- **Bonus D:** Degradation test - Remove half the documents from your vector store and re-run your evaluation set. Report how the agent's behaviour changes: does it fall back to web search? Does it hallucinate? Does it refuse more? Write up your findings.

13. What a Great Submission Looks Like

Based on past selection cycles, here is what separated the candidates we hired.

- **Tool descriptions:** Their tool descriptions were written for the LLM, not for a human. Each description told the model exactly when to use that tool and - crucially - when not to.
- **Loop clarity:** Their agent loop was under 80 lines of Python and readable top to bottom in under 3 minutes. Complexity was in the tools, not the loop.
- **Trace quality:** Their traces showed the agent reasoning, not just calling tools. You could see why it made a second call instead of a first call.
- **Honest evaluation:** Their evaluation report included failures, explained why each failure happened, and proposed a specific fix. Not “the model needs more data” - a specific diagnosis.
- **README quality:** Their README opened with what the project does, not how it works. It told a reader how to run it before explaining how it was built.
- **Curiosity:** They noticed something unexpected about their agent's behaviour during testing and wrote it down. That observation showed more than any accuracy number.

14. Frequently Asked Questions

Can I use LangChain or LangGraph?

You may use these libraries for utilities (e.g., text splitting, embeddings wrappers) but not as a drop-in replacement for your agent loop. Your agent loop - the part that decides which tool to call, handles the result, and decides what to do next - must be code you wrote and can explain. You will be asked to walk through it line by line during the interview.

What if a tool returns an error or times out?

Handle it explicitly. Your agent should catch tool errors, log them in the trace, and decide whether to retry (once), fall back to a different tool, or return a partial answer with a note about the failure. Unhandled exceptions that crash the agent will penalise your Safety score significantly.

My agent is calling the wrong tool for some questions. Should I fix it?

Document it in your evaluation report rather than overfitting your prompts to the test set. Understanding why the agent makes a wrong tool choice - and being able to explain it during the interview - is more valuable than a higher accuracy score that hides the underlying issue.

Do I need to use embeddings for the document search tool?

Not necessarily. A keyword-based BM25 retriever is acceptable if it retrieves relevant chunks reliably. What matters is that your search_docs tool returns the most relevant chunks for the query. Demonstrate that you verified retrieval quality manually before connecting it to the agent.

What if I cannot afford the API costs?

Use GPT-4o-mini or Claude Haiku during development - both are significantly cheaper than the full-size models and are sufficient for this task. Cache all LLM responses during development. If you are still concerned about cost, email the hiring contact and we will discuss options.

Build something you can fully explain.

The best submissions are the ones where the candidate understands every decision their agent makes.

Good luck.